HydrateAI

System Design Reference · LangGraph + Gemini + Firebase + Flutter

This document describes the technical architecture of the HydrateAI Android MVP. It covers the LangGraph AI agent design, Gemini API integration, Firebase data model, and Claude Code development workflow. Use this as the canonical engineering reference for onboarding, code reviews, and future feature planning.

1 · Architecture Overview

HydrateAI uses a four-layer architecture. The Flutter Android app handles all user interaction and sends events to Firebase Cloud Functions. The Cloud Functions act as a lightweight API gateway, routing requests to the LangGraph AI agent. The agent orchestrates calls to Gemini APIs for photo verification, coaching responses, and adaptive scheduling logic.

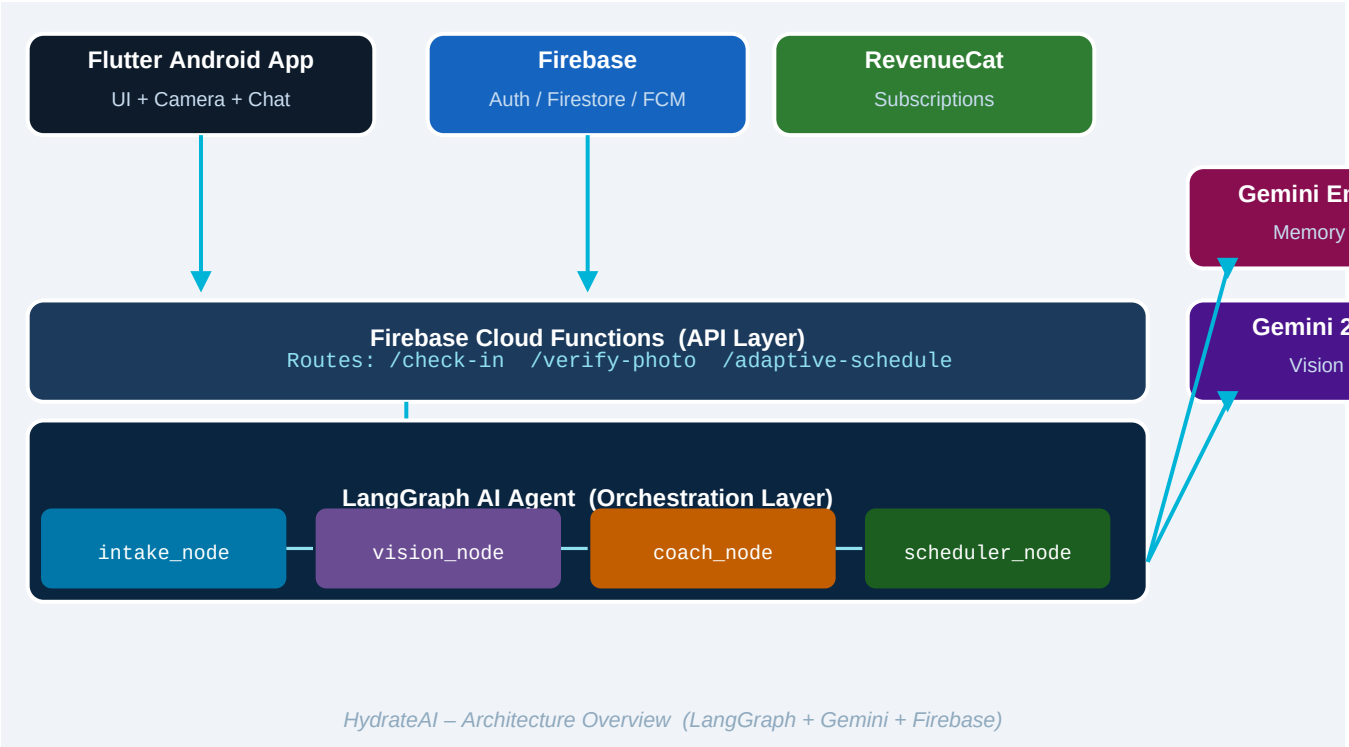
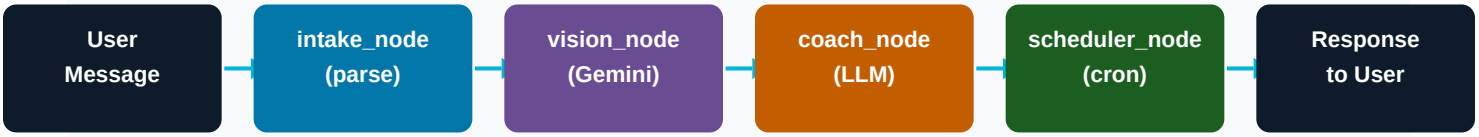


Figure 1 — Full system architecture

2 · LangGraph Agent Design

LangGraph models the AI coach as a stateful directed graph. Each user interaction passes through a defined sequence of nodes. This design makes each processing step testable in isolation, allows conditional routing (e.g. skip vision_node if no photo), and gives a clear audit trail for debugging AI responses.



LangGraph State Machine — each node is a pure Python function with typed state

Figure 2 — LangGraph state machine with typed AgentState

2.1 Node Responsibilities

Node	Responsibility	Gemini API Used
intake_node	Parse user message, classify intent (check-in / photo / question)	None
vision_node	Analyze photo → container type, volume, % remaining	gemini-2.0-flash (vision)
coach_node	Generate personalized AI coaching response, update streak	gemini-2.0-flash (text)
scheduler_node	Recalculate optimal next reminder time based on behavior history	None (rule-based)

2.2 Typed State Schema

All nodes share a single AgentState TypedDict passed through the graph. State is immutable between nodes — each node returns a partial update.

```
from typing import TypedDict, Optional

class AgentState(TypedDict):
user_id: str
message: str # raw user input
photo_url: Optional[str] # GCS URL if photo attached
intent: str # 'checkin' | 'photo' | 'question'
oz_logged: Optional[float]
ai_confidence: Optional[float]
coach_reply: Optional[str]
next_reminder: Optional[str] # ISO timestamp
streak_delta: int # +1 / 0 / -1
```

2.3 Conditional Graph Edges

LangGraph supports conditional routing. After intake_node classifies the intent, the graph branches: photo intents route through vision_node before coach_node; text check-ins skip directly to coach_node. This keeps each node lean and avoids unnecessary Gemini API calls.

```
from langgraph.graph import StateGraph, END

def route_after_intake(state: AgentState) -> str:
if state['intent'] == 'photo' and state.get('photo_url'):
return 'vision_node'
return 'coach_node'

graph = StateGraph(AgentState)
graph.add_node('intake_node', intake_node)
graph.add_node('vision_node', vision_node)
graph.add_node('coach_node', coach_node)
graph.add_node('scheduler_node', scheduler_node)
graph.add_conditional_edges('intake_node', route_after_intake)
graph.add_edge('vision_node', 'coach_node')
graph.add_edge('coach_node', 'scheduler_node')
graph.add_edge('scheduler_node', END)
app = graph.compile()
```

3 · Gemini API Integration

HydrateAI uses two Gemini capabilities: multimodal vision for photo verification and text generation for coaching messages. Both are called via the Google AI Python SDK (google-generativeai). The vision_node sends the bottle photo plus a structured prompt; the coach_node sends conversation history plus user context.

3.1 Photo Verification (vision_node)

```
import google.generativeai as genai
from google.generativeai.types import HarmCategory, HarmBlockThreshold

genai.configure(api_key=os.environ['GEMINI_API_KEY'])
model = genai.GenerativeModel('gemini-2.0-flash')

def vision_node(state: AgentState) -> dict:
img_data = fetch_image_from_gcs(state['photo_url'])
prompt = (
'Analyze this hydration container. Return JSON: '
'{container_type, total_oz, pct_remaining, oz_consumed, confidence}'
)
response = model.generate_content([prompt, img_data])
parsed = json.loads(response.text)
return {
'oz_logged': parsed['oz_consumed'],
'ai_confidence': parsed['confidence']
}
```

3.2 Coach Response (coach_node)

```
SYSTEM_PROMPT = """
You are an encouraging ADHD-friendly hydration coach.
Be brief (2-3 sentences). Reference the user's streak.
Never shame. Always motivate. Use their name.
"""

def coach_node(state: AgentState) -> dict:
context = load_user_context(state['user_id']) # from Firestore
user_prompt = build_prompt(state, context)
response = model.generate_content(
[SYSTEM_PROMPT, user_prompt],
generation_config={'temperature': 0.7, 'max_output_tokens': 150}
)
return {'coach_reply': response.text}
```

4 · Firebase Data Model

Firestore is used as the primary database. Collections are designed for real-time listener efficiency — the Flutter app subscribes to the user's document and streak doc directly. Cloud Functions write to logs

subcollections.

4.1 Firestore Collections

Collection: users/{user_id}

Field	Type	Notes
user_id	string	Firebase Auth UID
display_name	string	
timezone	string	e.g. America/New_York
daily_goal_oz	number	Default 64
subscription_tier	string	'free' | 'premium'
adhd_mode	boolean	Enables adaptive scheduling
onboarded_at	timestamp	

Subcollection: users/{user_id}/hydration_logs

Field	Type	Notes
log_id	string	Auto-generated
amount_oz	number	AI-estimated or manual
photo_url	string | null	GCS path
ai_confidence	number | null	0.0–1.0
source	string	'photo' | 'text' | 'manual'
logged_at	timestamp	

Document: users/{user_id}/stats/streak

Field	Type	Notes
current_streak	number	Days in a row meeting goal
best_streak	number	All-time best
last_completed_date	string	YYYY-MM-DD
consistency_score	number	0–100 weekly score
response_rate_7d	number	% check-ins answered

5 · Project Folder Structure

The repository uses a monorepo layout with three top-level packages: the Flutter app, the Firebase Cloud Functions backend, and the Python LangGraph agent. This separation allows independent testing and deployment.

```
hydrateai/
■■■ flutter_app/                  # Flutter Android app
■   ■■■ lib/
■   ■   ■■■ main.dart
■   ■   ■■■ screens/
■   ■   ■   ■■■ home_screen.dart
■   ■   ■   ■■■ chat_screen.dart
■   ■   ■   ■■■ onboarding_screen.dart
■   ■   ■■■ services/
■   ■   ■   ■■■ auth_service.dart
■   ■   ■   ■■■ firestore_service.dart
■   ■   ■   ■■■ notification_service.dart
■   ■   ■■■ models/
■   ■   ■   ■■■ user_model.dart
■   ■   ■   ■■■ hydration_log.dart
■   ■   ■■■ widgets/
■   ■■■ android/
■   ■■■ pubspec.yaml
■
■■■ functions/                    # Firebase Cloud Functions
■   ■■■ src/
■   ■   ■■■ index.ts              # Entry point (callable functions)
■   ■   ■■■ checkin.ts            # POST /check-in
■   ■   ■■■ verify_photo.ts       # POST /verify-photo
■   ■   ■■■ schedule.ts           # Adaptive cron trigger
■   ■■■ package.json
■   ■■■ tsconfig.json
■
■■■ agent/                        # LangGraph Python agent
■   ■■■ graph.py                  # StateGraph definition
■   ■■■ nodes/
■   ■   ■■■ intake_node.py
■   ■   ■■■ vision_node.py
■   ■   ■■■ coach_node.py
■   ■   ■■■ scheduler_node.py
■   ■■■ state.py                  # AgentState TypedDict
■   ■■■ prompts.py                # System + user prompts
■   ■■■ tests/
■   ■■■ requirements.txt
■
■■■ docs/                         # This document + ADRs
■■■ firebase.json
■■■ .env.example
```

6 · Claude Code Development Workflow

Claude Code (claude.ai/code) is used as the primary engineering assistant. Use the prompts below to initialize each layer of the stack. Run them in order for fastest path to a working MVP.

PROMPT 1 — Initialize Flutter project

```
Create a Flutter Android project called "hydrateai" with Firebase Auth, Firestore, Firebase Messaging, and RevenueCat. Set up the folder structure from the spec: screens/, services/, models/, widgets/. Create placeholder files with correct imports. Add google-services.json placeholder. Use Riverpod for state management.
```

PROMPT 2 — Firebase Cloud Functions scaffold

```
Create a Firebase Cloud Functions project in TypeScript. Add three callable functions: checkIn (receives user_id, message, optional photo_url), verifyPhoto (receives user_id, photo_url), and updateSchedule (receives user_id). Each function should call a Python Cloud Run service via HTTP. Add Firebase Admin SDK initialization and Firestore write helpers.
```

PROMPT 3 — LangGraph agent scaffold

```
Build a LangGraph agent in Python with AgentState TypedDict (user_id, message, photo_url, intent, oz_logged, ai_confidence, coach_reply, next_reminder, streak_delta). Create four nodes: intake_node, vision_node, coach_node, scheduler_node. Add conditional routing: photo intents go through vision_node, text intents skip to coach_node. Use google-generativeai SDK (gemini-2.0-flash) for vision_node and coach_node. Include a FastAPI wrapper as the entry point.
```

PROMPT 4 — Photo verification Gemini prompt

```
Write a vision_node function that takes a GCS image URL, downloads the image, sends it to Gemini 2.0 Flash with a structured prompt to return JSON: {container_type, total_oz, pct_remaining, oz_consumed, confidence}. Parse the JSON response, handle errors gracefully (fallback to 8 oz if parsing fails), and return the oz_consumed and confidence values to update AgentState.
```

PROMPT 5 — Flutter chat screen

```
Create a Flutter ChatScreen widget that displays an iMessage-style chat UI. User messages appear right-aligned in teal bubbles; AI messages appear left-aligned in dark navy bubbles. Include a bottom input bar with a text field and a camera attachment button. When the camera button is pressed, open the image picker, upload the image to Firebase Storage, and send the URL to the checkIn Cloud Function. Use StreamBuilder to listen to Firestore for real-time message updates.
```

7 · Scalability & Future Expansion

This architecture is intentionally designed to scale to HabitPilot (multi-habit platform) without a rewrite. The LangGraph agent is habit-agnostic — the AgentState can be extended with habit_type and any intake node can parse medication, sleep, or exercise check-ins using the same pattern.

Concern	Current (MVP)	Phase 3 (Platform)
Habits	Hydration only	Add field: habit_type to AgentState
Users	Firebase free tier	Firebase Blaze + caching layer
AI model	gemini-2.0-flash	Fine-tuned Gemini or private model
Platform	Android only	iOS via same Flutter codebase
Notifications	FCM push	FCM + Twilio SMS + WhatsApp
Monetization	RevenueCat direct	Add B2B / corporate wellness tier